

1. Callbacks

1.1. Funcionamiento básico

Una función de *callback* es una función que se pasa como parámetro a otra función, que la ejecutará cuando crea conveniente (normalmente, después de realizar un procesamiento).

Hemos estudiado los *callbacks* en algunas funciones, como la función **map**. Esta función recibe una función como parámetro. La función principal, **map**, recibe una función *callback* como parámetro. La tarea de **map** es recorrer los elementos de un *array* y ejecutar la función de *callback* sobre cada uno de dichos elementos.

En el caso de **map**, la ejecución de la función de *callback* es inmediata. Se dice que es una función **síncrona**: hasta que **map** no termine, no se sigue ejecutando la siguiente línea de programa. A continuación puedes ver un ejemplo:

```
let letras = ["a", "b", "c"];

let letrasMayusculas = letras.map(function(letra) {
  console.log(`Convirtiendo letra '${letra}'`);
  return letra.toUpperCase();
});

console.log("Esta línea se ejecuta después de que termine 'map'");
```

También hemos trabajado con *callbacks* en la gestión de eventos. Por ejemplo, la función **addEventListener** permite definir una función, un *callback*, que se ejecuta en respuesta a un evento determinado.

1.2. Programación asíncrona

La programación asíncrona consiste en utilizar funciones cuyo resultado no se devuelve de manera inmediata, sino pasado un tiempo. Normalmente, ese tiempo es debido a que hay que realizar algún tipo de procesamiento, como una operación de acceso a **disco** o una operación de acceso a la **red**. En esos casos, no es aceptable bloquear la ejecución del programa para esperar a que termine dicho proceso, sino que es más conveniente permitir que el programa continúe realizando tareas que no necesitan el resultado del proceso: cuando se complete el proceso, se generará una especie de **alarma** que el entorno de ejecución detectará y pasará a ejecutar la función de *callback* para procesar el resultado.

Una de las funciones más sencillas de utilizar para ilustrar el funcionamiento de la programación asíncrona es la función **setTimeout**. Esta función tiene la siguiente sintaxis:

```
setTimeout(FUNCION_CALLBACK, TIEMPO_ESPERA);
```

El primer parámetro de la función es una función de *callback*. Dicha función se ejecutará cuando pase el intervalo de tiempo definido en el segundo parámetro. Así, un ejemplo de uso sería

```
function saludar() {
  console.log("hola");
}

// Mostrará "hola" transcurridos dos segundos
setTimeout(saludar, 2000);
```

El código declarado después de la llamada a la función `setTimeout` se ejecutará **antes** que la función de *callback*:

```
function saludar() {  
    console.log("hola");  
}  
  
// Mostrará "hola" transcurridos dos segundos  
setTimeout(saludar, 2000);  
  
console.log("Esta línea se ejecuta ANTES de que se muestre por pantalla  
'hola'");
```

Normalmente, con *callbacks* se utilizan funciones anónimas:

```
// Mostrará "hola" transcurridos dos segundos  
setTimeout(function() {  
    console.log("hola");  
}, 2000);  
  
console.log("Esta línea se ejecuta ANTES de que se muestre por pantalla  
'hola'");
```

Siguiendo con el ejemplo anterior, si ejecutamos dos funciones `setTimeout` de manera independiente, su procesamiento se realiza **en paralelo**: ambos temporizadores comienzan prácticamente en el mismo instante. Por ejemplo:

```
// Mostrará "hola" transcurridos dos segundos  
setTimeout(function() {  
    console.log("hola");  
}, 2000);  
  
// Mostrará "adiós" transcurridos tres segundos  
setTimeout(function() {  
    console.log("adiós");  
}, 3000);
```

El programa se ejecuta y pone en marcha los dos temporizadores de manera casi instantánea (solo separados por un ciclo de procesador). A continuación, muestra por pantalla **hola** a los 2 segundos de su arranque y muestra **adiós** a los 3 segundos. Así, entre la aparición de ambos textos transcurre solo **un segundo**.

Si se desea lanzar un temporizador **después** de que termine el primero, es necesario ejecutarlos **en serie**. Para ello, hay que **arrancar el segundo temporizador dentro del *callback* del primero**. A esta técnica se la denomina **anidación**. Veremos un ejemplo de uso en la sección siguiente.

1.3. Inconvenientes de los *callbacks*

El uso de *callbacks* tiene un inconveniente relacionado con su anidación. Consideremos un ejemplo en el que queremos encadenar varios temporizadores para que se ejecuten en serie (uno a continuación de otro):

```
setTimeout(function() {  
  console.log("Primer mensaje después de 2 segundos.");  
  // Ponemos en marcha el segundo temporizador  
  setTimeout(function() {  
    console.log("Segundo mensaje, 3 segundos después del primero.");  
    // Ponemos en marcha el tercer temporizador  
    setTimeout(function() {  
      console.log("Tercer mensaje, 1 segundo después del segundo.");  
    }, 1000);  
  }, 3000);  
}, 2000);
```

El código anterior es complicado de interpretar. Además, conforme profundizamos en la anidación vemos que el código se va desplazando a la derecha. A esto se le conoce como *infierno de callbacks* (*callback hell*) o *pirámide de la perdición* (*pyramid of doom*).

En segundo lugar, el uso de *callbacks* no sigue el paradigma funcional: las funciones no devuelven resultados, sino que simplemente ejecutan código en su interior. Esto puede crear problemas de legibilidad del código y de uso de variables. Por ejemplo:

```
var resultado = "";  
  
setTimeout(function() {  
  resultado = "hola";  
}, 2000);  
  
console.log(resultado); // Cadena vacía: 'resultado' se actualiza cuando se  
ejecute el callback
```

Por último, el uso de *callbacks* puede crear problemas si se utilizan dentro de métodos de objetos. Consideremos el siguiente ejemplo:

```
let usuario = {  
  nombre: "Laura",  
  saludarConRetardo: function() {  
    setTimeout(function() {  
      // Función convencional: define 'this', que se ejecuta en el contexto  
      // de una función anónima  
      // (no tiene referencia ni implícita ni explícita al objeto sobre el  
      // que se ejecuta).  
      // 'this' en este caso apunta al objeto global  
      console.log(`Hola, soy ${this.nombre}`);  
    }, 2000);  
  }  
};  
  
usuario.saludarConRetardo();  
// Cuando pasen dos segundos se mostrará: 'Hola, soy undefined'
```

Para que el código anterior funcione, es necesario que **this** apunte al objeto que queremos. Una solución es utilizar una función flecha:

```
let usuario = {
  nombre: "Laura",
  saludarConRetardo: function() {
    setTimeout(() => {
      // Función flecha: no tiene 'this'
      // Por tanto, 'this' se coge del contexto léxico
      // (cuerpo de la función 'saludarConRetardo')
      console.log(`Hola, soy ${this.nombre}`);
    }, 2000);
  }
};

usuario.saludarConRetardo();
// Cuando pasen dos segundos se mostrará: 'Hola, soy Laura'
```